

Lab3

Henrique Machado, Daniel Fernandes

March 2026

Exercise 1

Training Comparison

Looking at the training cumulative cost plot (Figure 1, left), **PredictiveAgent** accumulates cost linearly throughout all 5000 episodes, and we observe that its slope never changes since it has no learning mechanism. **Q-Learning** and **SARSA** start with a similar slope to the baseline during early episodes ($\epsilon \approx 1.0$, mostly random actions), but both show a clear reduction in slope around episode 1500-2000, coinciding with ϵ decaying below ~ 0.2 and exploitation beginning to dominate. By episode 5000, both RL agents have accumulated significantly less cost than the PredictiveAgent.

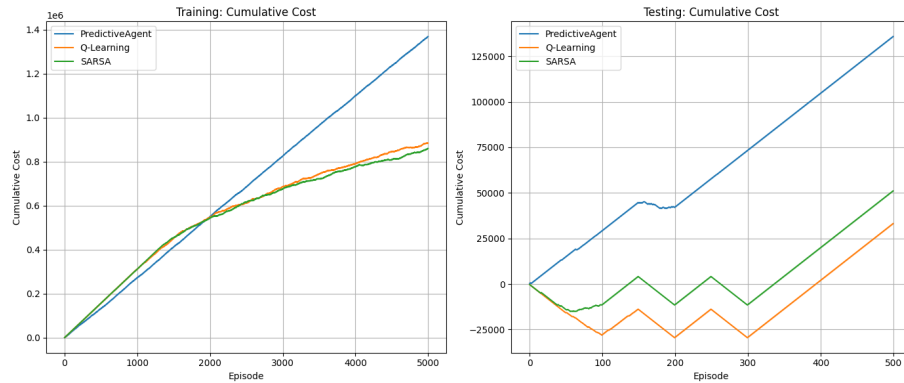


Figure 1: Training (left) and testing (right) cumulative cost for all agents over 5000 episodes.

Test Comparison

At test time the difference is obvious. Both RL agents achieve negative cumulative cost, meaning they win more than they lose, while the PredictiveAgent

Agent	Win Rate	Avg Cost	Avg Steps
PredictiveAgent	~6%	~270	~746
Q-Learning	~39%	~66	~791
SARSA	~33%	~102	~797

Table 1: Test performance comparison across agents.

grows linearly positive, consistently losing. Q-Learning and SARSA perform similarly, with results varying between runs due to stochasticity.

Policy Differences and Convergence

Q-Learning (off-policy) learns toward the optimal greedy policy directly by bootstrapping with $\max_{a'} Q(s', a')$, regardless of what action was actually taken. SARSA (on-policy) bootstraps with the action actually chosen by ϵ -greedy, so its learned values reflect the behaviour policy including exploration noise, making it slightly more conservative.

In terms of convergence, both algorithms improve at roughly the same rate and their curves are nearly indistinguishable in training. The convergence is driven primarily by ϵ decay rather than the number of episodes — the agents improve when they start exploiting, not simply from seeing more data. The PredictiveAgent shows no convergence at all, confirming that a fixed heuristic cannot adapt to the diverse fish dynamics in this environment.

Exercise 2

2.1 - Multi-seed Robustness

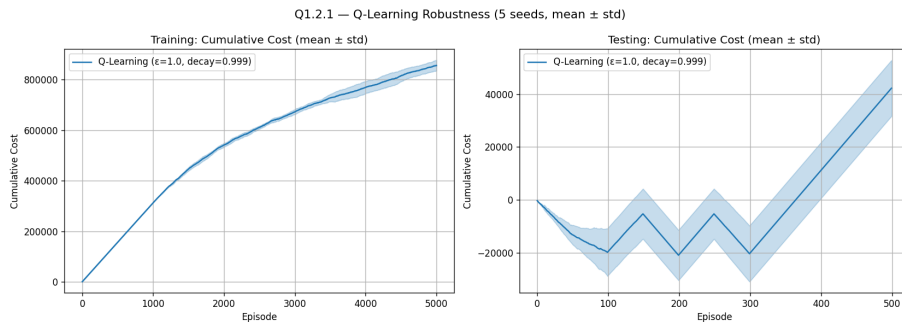


Figure 2: Q-Learning training and testing cumulative costs over 5 random seeds (mean $\pm$ std).

As shown in Figure 2, the training cumulative cost is highly consistent across all 5 seeds: the ± 1 standard deviation band is extremely narrow throughout all

5000 episodes, and the individual curves are nearly indistinguishable. This indicates that Q-Learning training is stable and reproducible, meaning the learning dynamics are not sensitive to the random seed. The testing cumulative cost shows greater variance, with the standard deviation band widening noticeably from episode ~ 300 onwards. This is expected: the test suite is ordered by fish difficulty (the final fish types, Tuna, Shark, Legend, and The Glitch, are substantially more erratic), and different seeds converge to slightly different policies that handle the hardest fish with differing success. The growing variance at test time therefore reflects genuine policy uncertainty on difficult fish rather than algorithmic instability. Overall, Q-Learning is a robust algorithm for this environment: its training behaviour is highly consistent across seeds, and the test variance is explained by environmental difficulty rather than unreliable convergence.

2.2 - Is the Final Policy Optimal?

The final policy is **not guaranteed to be optimal**, for the following reasons:

1. **Discretization loss.** The continuous state is binned into a 20×10 grid (relative position $\times$ bar velocity). States that differ in the continuous space but fall in the same bin are treated identically, so the tabular policy cannot represent fine-grained distinctions that a truly optimal policy might require.
2. **Incomplete state representation.** Fish velocity is not included in the state. An optimal controller would benefit from knowing the fish’s current direction of movement in order to anticipate its future position. Without this, the policy can only react to the current relative position, which is a structural limitation.
3. **Indirect convergence evidence.** The training cumulative cost curve flattens around episode 3000-4000, suggesting Q-values have largely stabilised. However, flattening of cumulative cost indicates the policy has stopped improving noticeably within the given budget; it does not prove convergence to the true optimal Q^* .
4. **Stochastic environment.** Fish dynamics include random Gaussian noise and jump events. Even the true optimal policy cannot achieve cost -1 at every step, so there exists a natural performance floor that cannot be eliminated by any policy.

Despite these limitations, Q-Learning clearly outperforms the PredictiveAgent baseline at test time (lower cumulative cost), confirming it has found a better-than-heuristic policy. Optimality in the strict sense cannot be claimed, but the policy is a strong practical solution within the constraints of tabular RL and the chosen state representation.

2.3 - Impact of Exploration Parameters

(a) Initial Epsilon

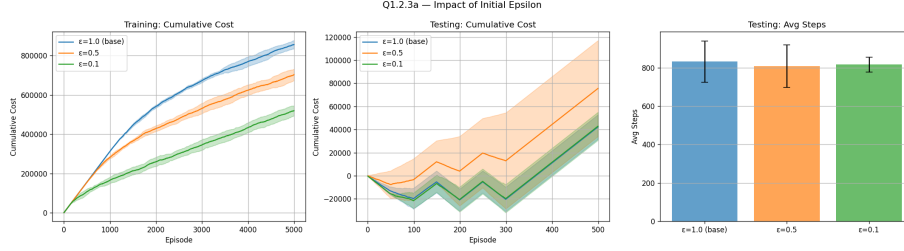


Figure 3: Impact of initial epsilon on training cumulative cost (left), testing cumulative cost (centre), and average steps per episode (right). Results are averaged over 5 seeds (mean $\pm$ std).

As shown in Figure 3, the initial value of ϵ affects training and testing performance differently. During **training**, $\epsilon = 0.1$ accumulates the lowest cumulative cost because the agent exploits its current (partial) policy immediately and rarely takes random actions. However, this is not indicative of better learning; it simply means fewer exploratory (and thus more costly) random actions are taken. During **testing**, $\epsilon = 0.5$ produces the worst and most variable performance. A moderate initial epsilon provides neither enough exploration to cover the state space thoroughly nor enough exploitation to reinforce good actions early on. Both $\epsilon = 1.0$ and $\epsilon = 0.1$ achieve similarly low test cumulative costs, but $\epsilon = 1.0$ yields a tighter standard deviation band, indicating more consistent results across seeds. Starting fully random ensures broad state-space coverage in early episodes, producing a more complete Q-table.

Average steps are nearly identical across all three configurations (~ 800 – 825), meaning the episode length is not significantly affected by the initial ϵ alone. In summary, starting with high ϵ (1.0) is the safer choice: it guarantees broad initial state-space coverage. Too low an initial ϵ risks premature exploitation of a suboptimal policy, while an intermediate value yields the worst of both worlds.

(b) Epsilon Decay

Figure 4 shows that the decay rate has a larger and more decisive impact than the initial ϵ value.

Fast decay (0.99): ϵ reaches its minimum (~ 0.01) at around episode 460. The agent quickly transitions to exploitation, resulting in much lower training cumulative cost. At test time it performs well with tight variance and achieves the lowest average steps (~ 765), indicating a more decisive and consistent policy.

Slow decay (0.9995): ϵ remains above 0.1 until approximately episode 4600, so the agent acts near-randomly for the majority of training. This pro-

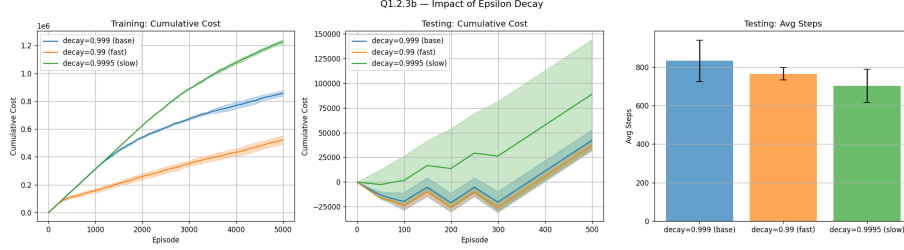


Figure 4: Impact of epsilon decay rate on training cumulative cost (left), testing cumulative cost (centre), and average steps per episode (right). Results are averaged over 5 seeds (mean $\pm$ std).

duces the highest training cumulative cost ($>1.2 \times 10^6$) and catastrophically high test cumulative cost with enormous variance. Within 5000 episodes the Q-table never receives sufficient exploitation updates to consolidate a reliable policy.

Base decay (0.999): A good compromise between the two extremes. It allows enough early exploration to build a representative Q-table while converging to stable exploitation within the training budget. In summary, the decay rate is the most critical exploration hyperparameter in this setting: too slow a decay prevents the policy from ever materialising within the training budget, while too fast a decay risks missing parts of the state space. The base configuration (0.999) balances these considerations well for 5000 training episodes.

2.4 - Single Fish Type vs. Mixed Training



Figure 5: Sturgeon-only training vs. mixed training: training cumulative cost (left), testing cumulative cost on all fish types (centre), and average steps (right). Results are averaged over 5 seeds (mean $\pm$ std).

As shown in Figure 5, the two training regimes diverge sharply during training but converge at test time.

Training cost: Sturgeon-only training exhibits a distinctive inverted-U shape: the cost rises initially during random exploration, then drops sharply

into highly negative territory (reaching $\sim -400\,000$ by episode 5000), indicating the agent has nearly mastered the Sturgeon fish. Mixed training plateaus at $\sim +850\,000$, reflecting the harder average difficulty across all ten fish types and the fact that the policy must remain general rather than specialising.

Testing on all fish: Despite the large difference in training performance, the test cumulative cost curves are **very similar** between the two agents, with Sturgeon-only training performing only marginally better. The Sturgeon-trained agent generalises to other fish types reasonably well. This is explained by the state representation: the discretized state (relative position and bar velocity) is fish-agnostic, encoding the control problem without encoding fish identity. The fundamental skill of keeping the bar near a moving target transfers across fish types.

Average steps: Sturgeon-only training yields slightly lower average steps (~ 750 vs. ~ 825) with a tighter error bar, indicating more consistent and decisive episode resolution.

Conclusion: Training on a single fish type does generalise to others in this environment, because the underlying control problem is shared. However, mixed training is more robust for deployment across unknown fish types: it exposes the agent to a wider variety of dynamics during learning, producing a policy that is less specialised but more evenly capable across the full difficulty spectrum.

Exercise 3

The base Q-Learning agent state representation consists of two features: the relative position `rel_pos = fish_y - (bar_y + 40)` and the bar velocity `bar_vel`. While this captures the current error and bar momentum, it contains no information about where the fish is *going*. A fish moving rapidly upward requires a different response than a stationary fish at the same position.

Since `fishing_logic.py` cannot be modified and `get_state()` does not expose fish velocity, it is estimated internally from two consecutive observations:

$$\hat{v}_{\text{fish}}(t) = \text{fish_y}(t) - \text{fish_y}(t - 1)$$

This estimate is reset at the start of each episode to avoid carrying over stale velocity from the previous episode, and discretized into 7 bins over the range $[-30, +30]$, giving a total state space of $20 \times 10 \times 7 = 1400$ states.

Agent	Win Rate	Avg Cost	Avg Steps
Q-Learning	33.0%	107.5	811.5
Improved Q-Learning	38.8%	69.4	784.6

Table 2: Test performance comparison between base and improved Q-Learning agents.

The improved agent outperforms the base on all metrics. The reduction in average steps indicates the agent wins faster, suggesting it acts more proactively

by anticipating fish movement rather than reacting to it. Note that $\hat{v}_{\text{fish}}$ is a noisy estimate, yet the additional feature still provides a clear net benefit.